



National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

Department of Computing

CS220: Database Systems

Class: BSAI – 2A

Lab 10: Correlated Nested Queries

CLO-2: Formulate SQL queries to retrieve information from a relational database.

Date: 16 April 2026

Time: 10:00 - 13:00



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

Introduction

- A correlated subquery is one where the inner query depends on values provided by the outer query. This means the inner query is executed repeatedly, once for each row that might be selected by the outer query.

Objectives

After performing this lab, students should be able to:

1. Understand and formulate Correlated Nested Queries

Tools/Software Requirement

- MySQL Community Server
- MySQL Workbench

Description

Nested Queries/Subqueries

A nested/subquery is a SQL query nested inside a larger query, such inner-outer queries are called nested queries

A subquery may occur in:

- A SELECT clause
- A FROM clause
- A WHERE clause

Rule of thumb: avoid writing nested queries when possible; keep in mind that sometimes it's impossible

CORRELATED NESTED/SUBQUERIES

A correlated subquery is one where the inner query depends on values provided by the outer query. This means the inner query is executed repeatedly, once for each row that might be selected by the outer query. For example,

Query: **Find the names of students enrolled in any classes**



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

```
select distinct s.sname  
from student s  
where Exists (select * e.snum  
from enrolled e where s.snum = e.snum)
```

The subquery in this SELECT statement cannot be resolved independently of the main query. Notice that the outer query specifies that rows are selected from the student table with an alias name of s. The inner query compares the snum (student name) column of the enrolled table to the same column of the student table. The subquery's results are correlated with each individual row of the main query – thus, the term correlated subquery.

When a subquery uses the EXISTS operator, the subquery functions as an existence test. The WHERE clause of the outer query tests for the existence of rows returned by the inner query. The subquery does not actually produce any data; rather, it returns a value of TRUE or FALSE.

The general format of a subquery WHERE clause with an EXISTS operator as shown below. Note that the NOT operator can also be used to negate the result of the EXISTS operator.

WHERE [NOT] EXISTS (subquery)

Subqueries using an EXISTS operator are a bit different from other subqueries, in the following ways:

1. The keyword EXISTS is not preceded by a column name, constant, or other expression.
2. The SELECT clause list of a subquery that uses an EXISTS operator almost always consists of an asterisk (*). This is because there is no real point in listing column names since you are simply testing for the existence of rows that meet the conditions specified in the subquery.
3. The subquery evaluates to TRUE or FALSE rather than returning any data.
4. A subquery that uses an EXISTS operator will always be a correlated subquery.

The EXISTS operator is very important, because there is often no alternative to its use. All queries that use the IN operator or a modified comparison operator (=, <, >, etc. modified by ANY or ALL) can be expressed with the EXISTS operator. However, some queries formulated with EXISTS cannot be expressed in any other way. For example,

Query: **Find the names of students enrolled in any classes**

```
select distinct s.sname
```



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

```
from student s
where s.snum in (select e.snum
from enrolled e);
```

```
select distinct s.sname
from student s
where Exists (select * e.snum
from enrolled e where s.snum = e.snum)
```

The NOT EXISTS operator is the mirror-image of the EXISTS operator. A query that uses NOT EXISTS in the WHERE clause is satisfied if the subquery returns no rows.

Lab Task

Use the following database schema to fulfill the following information needs:

Student (snum: integer, sname: char(30), major: char(25), level: char(2), age: integer)

Faculty (fid: integer, fname: char(30), deptid: integer)

Class (cname: char(40), meets_at: char(20), room: char(10), fid: integer | fid REFS Faculty.fid)

Enrolled (snum: integer, cname: char(40) | snum REFS student.snum, cname REFS class.name)

Write SQL expressions for the following queries and execute them:

1. Find the name of faculty members who do not teach any course.

CODE:

```
Select fname from faculty
Where fid NOT IN(
select distinct fid from class
);
```



OUTPUT:

Result Grid	
	fname
▶	John Smith
	Dr Khan

- Find the names of faculty members that have taught classes only in room R128.

CODE:

```
SELECT fname
FROM Faculty f
WHERE f.fid IN (
    SELECT fid FROM Class
    GROUP BY fid
    HAVING COUNT(DISTINCT room) = 1
    AND MAX(room) = 'R128'
);
```

OUTPUT:

Result Grid	
	fname
▶	Ivana Teach



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

3. Retrieve the snum and sname of students who have taken classes from both 'Ivana Teach' and 'Linda Davis'.

CODE:

```
SELECT s.snum, s.sname
FROM Student s
WHERE s.snum IN (
    SELECT e.snum
    FROM Enrolled e
    JOIN Class c ON e.cname = c.cname
    JOIN Faculty f ON c.fid = f.fid
    WHERE f.fname = 'Ivana Teach'
)
AND s.snum IN (
    SELECT e.snum
    FROM Enrolled e
    JOIN Class c ON e.cname = c.cname
    JOIN Faculty f ON c.fid = f.fid
    WHERE f.fname = 'Linda Davis'
);
```

OUTPUT:

Result Grid		
	snum	sname
▶	1	Ali
	5	Usman
✱	NULL	NULL



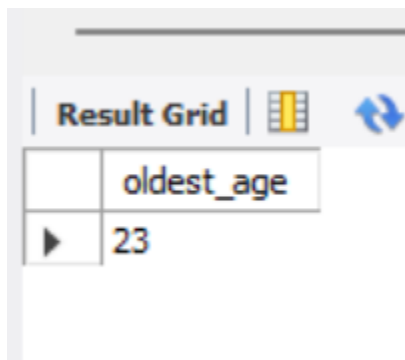
National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

4. Find the age of the oldest student(s) who is enrolled in a course taught by Ivana. Teach.

CODE:

```
SELECT MAX(s.age) AS oldest_age  
FROM Student s  
Where s.snum IN(  
Select s.snum  
from student s  
JOIN Enrolled e ON s.snum = e.snum  
JOIN Class c ON e.cname = c.cname  
JOIN Faculty f ON c.fid = f.fid  
WHERE f.fname = 'Ivana Teach'  
)
```

OUTPUT:



Result Grid	
	oldest_age
▶	23



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

5. Find the name of faculty members that do not teach to class 'database systems'.

CODE:

```
SELECT fname  
FROM Faculty  
WHERE fid NOT IN (  
    SELECT fid  
    FROM Class  
    WHERE cname = 'Database Systems'  
);
```

OUTPUT:

Result Grid	
	fname
▶	Ivana Teach
	Linda Davis
	John Smith
	Dr Khan



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

6. Find the name of faculty member, department who has taught the maximum number of distinct classes.

CODE:

```
SELECT f.fname, f.deptid
FROM Faculty f
WHERE (
    SELECT COUNT(DISTINCT c.cname)
    FROM Class c
    WHERE c.fid = f.fid
) = (
    SELECT MAX(class_count)
    FROM (
        SELECT COUNT(DISTINCT cname) AS class_count
        FROM Class
        GROUP BY fid
    ) temp
);
```

OUTPUT:

Result Grid			Filter Rows:
	fname	deptid	
▶	Ivana Teach	10	
	Linda Davis	20	



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

7. Find the names of all classes and their enrollment strength that have enrollment greater than 5.

CODE:

```
SELECT c.cname
FROM Class c
WHERE (
    SELECT COUNT(*)
    FROM Enrolled e
    WHERE e.cname = c.cname
) > 5;
```

OUTPUT:

Result Grid	
	cname
*	NULL

8. Find the names of all students who are enrolled in two classes that meet at the same time.

CODE:

```
SELECT s.sname
FROM Student s
WHERE EXISTS (
    SELECT *
```



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

FROM Enrolled e1, Enrolled e2, Class c1, Class c2

WHERE e1.snum = s.snum

AND e2.snum = s.snum

AND e1.cname = c1.cname

AND e2.cname = c2.cname

AND c1.meets_at = c2.meets_at

AND c1.cname <> c2.cname

);

OUTPUT:

Result Grid	
	sname
▶	Ali
	Usman

Deliverables

Complete your lab tasks in SQL workbench and submit a word file in with queries along with the screenshots of the results to all the questions attempted. Upload it on LMS. The marking will be based on viva/lab task submission.